Models of computation

Advanced Data Structures – Summer 2026

Ragnar {Groot Koerkamp}, Stefan Walzer, Stefan Hermann

2026-04-20 Mon 14:00

Today

- Why do we care about models?
- Big-O notation
- The complexity of sorting
- Properties data structures
- The word RAM model
  - Variations
  - Limitations

Motivation

Motivation

We want to design fast algorithms, but when is an algorithm *fast*?

- Implement it, and *measure* the time.
  - Extrapolate experiments to predict performance on larger .
  - May or may not give understanding.
  - Does not provide lower bounds.
- Analyse exactly what happens on the CPU.
  - Unwieldy; they are massive black boxes.
- Instead: use a simplified abstract *model* of the CPU.
  - Allows exact lower and upper bounds.

Big-O notation (recap)

Big-O notation (recap)

Big-, or notation relates the asymptotic growth of two abstract functions to each other.

Definition 1: *Big-O*.

Given two functions , we say that

when there exists constants and such that for all sufficiently large :

Example 1.

Definition 2: ***Extended notation*.**

- is like :
- is like :
- is like :
- is a stronger :
- is like :
- is like :

Quiz time!

What is the complexity of sorting?

What is the complexity of sorting?

- ?
- ?
- ?
- ?
- Linear ?
- Constant ?
- Something else?

What is the complexity of sorting?

All of the above!

- - Minimum number of *comparisons* is .
  - Merge sort indeed uses of them.
  - Or QuickSort, with comparisons *in expectation*.
- - Expected complexity of Van Emde-Boas trees

  (van Emde Boas 1975).
- - Fusion trees (Fredman and Willard 1993)
- ,
  - Fastest *randomized* algorithm

  (Han and Thorup, n.d.)

- Linear
  - Linear *space*? Sure!
  - Counting sort when all values are .
  - Radix sort on 64-bit integers.
  - Randomized when

    (Andersson et al. 1995)
- Constant
  - makes bounded.
  - There's only so many atoms in the universe.

ALGORITHM 64
QUICKSORT
C. A. R. HOARE
Elliott Brothers Ltd., Borehamwood, Hertfordshire, Eng.

```
procedure  quicksort (A,M,N);  value M,N;
             array A;  integer M,N;
comment  Quicksort is a very fast and convenient method of
sorting an array in the random-access store of a computer. The
entire contents of the store may be sorted, since no extra space is
required. The average number of comparisons made is 2(M−N) ln
(N−M), and the average number of exchanges is one sixth this
amount. Suitable refinements of this method will be desirable for
its implementation on any actual computer;
begin      integer I,J;
           if M < N then begin partition (A,M,N,I,J);
                                quicksort (A,M,J);
                                quicksort (A, I, N)
                          end
end        quicksort
```

Figure 1: Algorithm 64: Quicksort (Hoare 1961). RIP Tony Hoare.

"Complexity"

"Complexity"

- Lower or upper bound?
  - Complexity of a *problem* is at least .
  - Complexity of an *algorithm* at most .
  - "The complexity": matching lower and upper bound.
- Which metric?
  - "Operations"
  - Comparisons
  - Memory accesses
  - Memory touched
  - Wall time
  - Space usage
- What input type?
  - Integers
  - Floats
  - Abstract "*objects*"
- What input properties?
  - (Uniform?) random
  - Worst-case
- What algorithmic properties?
  - Deterministic
  - Randomized

Properties of data structures

Space usage

Let be the minimum space required to represent some data.

Definition 3: *Compact data structure*.

A *compact* data structure uses bits of space.

Definition 4: *Succinct data structure*.

A *succinct* data structure uses bits of space.

Example: many Rank & Select data structures are succinct.

Definition 5: *Implicit data structure*.

An *implicit* data structure uses bits of space.

Example: bit arrays.

Dynamic data structures

Definition 6: *Dynamic data structure*.

Allows mutating the structure.

Example: hash tables.

Definition 7: *Static data structure*.

Build once, then read-only queries.

Example: minimal perfect hash functions.

Definition 8: *Incremental data structure*.

Insert-only; deletions not allowed.

Example: bloom filters.

Running time

From stronger to weaker:

Definition 9: *Worst-case running time*.

An algorithm has *worst-case* running time when for *every possible* input, it is guaranteed to finishes in time.

Definition 10: *Expected running time*.

An algorithm has *expected* running time when for *every possible* input, the *expected* running time is .

Definition 11: *Running time with high probability*.

An algorithm has running time *with high probability* (w.h.p.) when for *every possible* input, the running time is with probability .

Running time

Definition 12: *Amortized running time*.

An operation on a data structure has *amortized* running time when for *every possible* sequence of operations with amortized complexities , the total running time is .

Example: pushing on a vector is worst-case , but amortized .

Models of computation

The RAM model

Definition 13: *Random-access machine (RAM) model*.

Models a machine that has an *infinite* list of *registers*.

- Each register can store an **unbounded (!)** natural number!
- Each register has an *address*.
- Retrieving the register corresponding to an address takes constant time.

Question 1.

Any problems with this?

Infinite memory, infinite registers, and constant-time access are strong assumptions.

The *real* RAM model

Definition 14.

Like the RAM model, but each register can store a *real* number.

The *word* RAM model

Definition 15.

Like the RAM model, but operates on *words* of bits.

- Registers are also bits.
- Constant-time -bit arithmetic operations:
  - bit-operations, addition, multiplication.

Evolved with CPUs to include: https://en.wikipedia.org/wiki/X86_Bit_manipulation_instruction_set

- Popcount (`popcnt`), count trailing zeros (`tzcnt`, BMI1)
- Parallel bit deposit (`pdep`), parallel bit extract (`pext`, BMI2)

Most commonly used model.

Question 2.

Any remaining problems with *this*?

Word RAM still assumes constant-time access. Constant time -bit multiplication?

We must talk about

We must talk about

- In practice, is constant.
- But the problem size grows to infinity.
- What if ? Then we can't even represent ?

We must talk about w

- In practice, $w = 64$ is constant.
- But the problem size n grows to infinity.
- What if $n > 2^w$? Then we can't even represent n?

Solution: we assume

$$\lg n \leq w.$$

This has far-reaching implications!

We must talk about w

- In practice, $w = 64$ is constant.
- But the problem size n grows to infinity.
- What if $n > 2^w$? Then we can't even represent n?

Solution: we assume

$$\lg n \leq w.$$

This has far-reaching implications!

- The complexity of pop-counting n bits is **sub-linear** $O(n/\lg n)$!
- Fusion trees can binary search on $O(\sqrt{w})$ w-bit integers in *constant* $O(1)$ time!

The pointer model

Definition 16: *Pointer model*.

RAM-model without the RAM:

- Registers can only be accessed via direct pointers.

Used in the analysis of some heap-based priority queues.

The cell-probe model

Definition 17: *Cell-probe model*.

Like the RAM-model, but all operations apart from accessing memory are free.

- Typically, an infinite cache is assumed, so that each address only has to be read once.
- Used for lower bounds on the number of addresses that must be read to solve some task, and thus a lower bound on RAM-model complexity.

The RAM-model is unphysical

The RAM-model is unphysical

- Infinite memory with constant time access is fundamentally not possible:
  - Each bit takes some small volume to store.
  - Signals can only travel at the speed of light .
  - In time , we can read from a sphere of radius , containing

Observation 1: *Latency*.

The *latency* of a uniform random access into a memory of size is at least

The RAM-model is unphysical (2)

The RAM-model is unphysical (2)

- Black holes have only *quadratic* mass !
  - Filling a volume with cubic information is *impossible*.
  - But we are far away from the constant.
- If each bit of memory requires some energy, we need to dissipate heat through of surface area.
  - This is an actual bottleneck for CPUs! They are mostly flat (2.5D) for a reason!
    - AMD now has "3D V-Cache" which vertically stacks caches.

Observation 2: *Black holes and cooling (handwavy)*.

We can store/cool at most bits in a sphere of radius , and so the *latency* of a uniform random access into a memory of size is at least

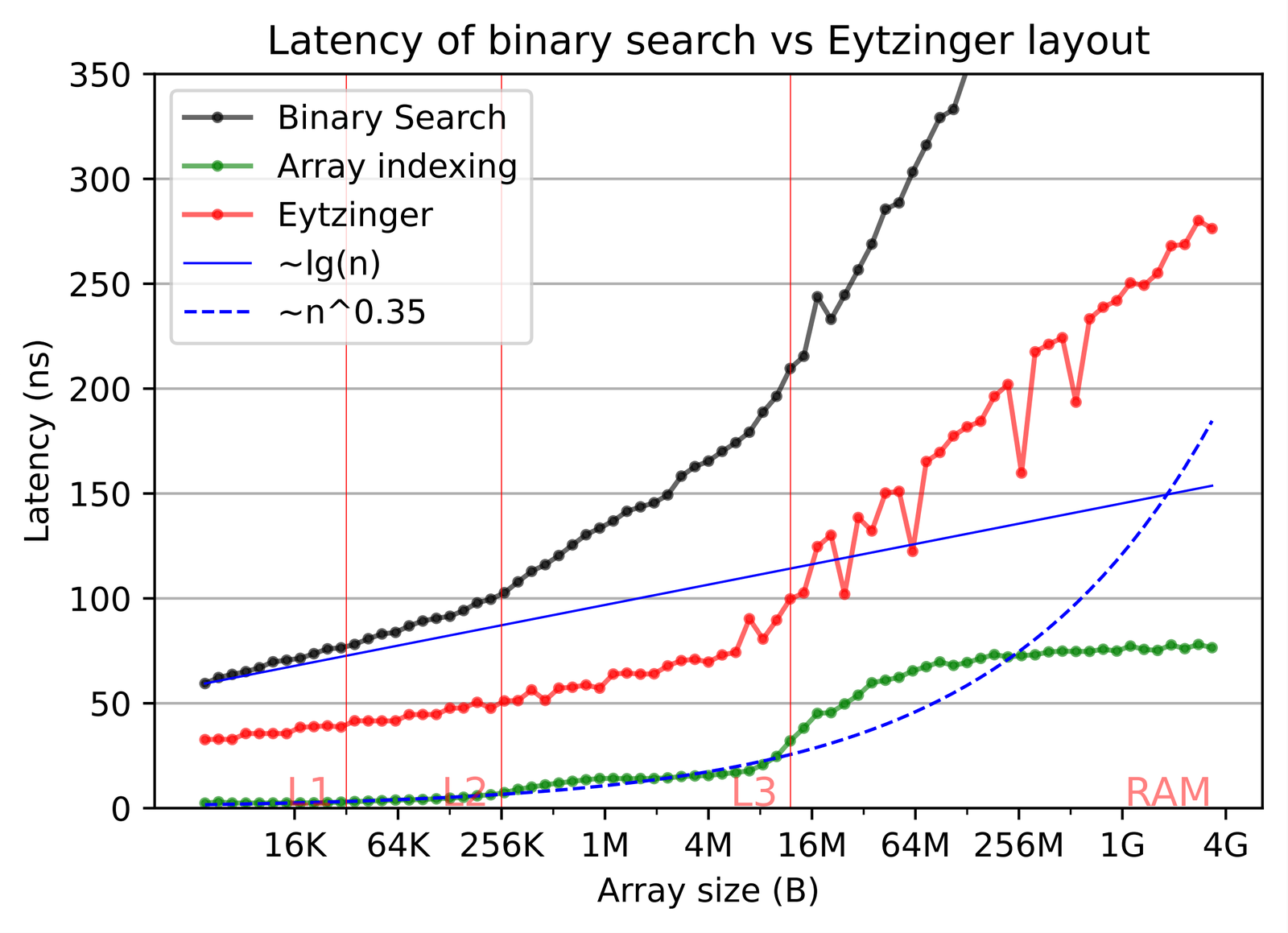
- See this blog: The Myth of RAM

The RAM-model is unrealistic

- Green: latency of array indexing.
  - Clearly not !
  - More like (dashed blue)
- Black: latency of binary search.
  - Clearly not (solid blue)
- Red: binary search with heap-layou
  - slower than just indexing?!

.

- **TODO:** -complexity model!



The I/O-complexity

Introduced by Aggarwal and Vitter (1988).

Definition 18: *I/O-complexity*.

Like the cell-probe model, the I/O-complexity only counts I/O operations.

- Fast and free *internal* memory of words.
- Slow (infinite) *external* memory.
- Count I/O-operations between them go in *blocks* of words.

Theorem 1: *I/O-complexity of sorting*.

Sorting word-sized integers has an I/O-complexity lower-bound of and external merge-sort achieves this.

Takeaways

Observation 3.

We need precisely defined models to theoretically analyse algorithms.

Observation 4.

Models approximate reality; some do so better than others.

The rest of this course will mostly use the **word RAM model** to analyse both theoretically and practically efficient algorithms.

Further reading

- These slides: curiouscoding.nl/teaching
- Algorithmica on binary search:
  - https://en.algorithmica.org/hpc/data-structures/binary-search/
  - Great resource for understanding and optimizing for modern CPUs
- Paper: Array layouts for comparison-based searching (Khuong and Morin 2017)
- Blog: The Myth of RAM
  - https://www.ilikebigbits.com/2014_04_21_myth_of_ram_1.html
- My P99 talk on binary search
  - https://curiouscoding.nl/slides/p99/
  - recording on youtube
- Aggarwal and Vitter's slides on I/O-complexity:
  - https://jshun.csail.mit.edu/6886-s19/lectures/lecture5-1.pdf

Next week: Bitvectors, Rank, & Select

Bibliography

Aggarwal, Alok, and S. Vitter Jeffrey. 1988. “The Input/Output Complexity of Sorting and Related Problems.” *Communications of the Acm* 31 (9): 1116–27. https://doi.org/10.1145/48529.48535.

Andersson, Arne, Torben Hagerup, Stefan Nilsson, and Rajeev Raman. 1995. “Sorting in Linear Time?” In *Proceedings of the Twenty-Seventh Annual Acm Symposium on Theory of Computing - Stoc ’95*, 427–36. Stoc ’95. ACM Press. https://doi.org/10.1145/225058.225173.

Emde Boas, P. van. 1975. “Preserving Order in a Forest in Less than Logarithmic Time.” In *16Th Annual Symposium on Foundations of Computer Science (Sfcs 1975)*. IEEE. https://doi.org/10.1109/sfcs.1975.26.

Fredman, Michael L., and Dan E. Willard. 1993. “Surpassing the Information Theoretic Bound with Fusion Trees.” *Journal of Computer and System Sciences* 47 (3): 424–36. https://doi.org/10.1016/0022-0000(93)90040-4.

Han, Yijie, and M. Thorup. n.d. “Integer Sorting in O(N√(Log Log N)) Expected Time and Linear Space.” In *The 43rd Annual Ieee Symposium on Foundations of Computer Science, 2002. Proceedings.*, 135–44. Sfcs-02. IEEE Comput. Soc. https://doi.org/10.1109/sfcs.2002.1181890.

Hoare, C. A. R. 1961. “Algorithm 64: Quicksort.” *Communications of the Acm* 4 (7): 321. https://doi.org/10.1145/366622.366644.

Khuong, Paul-Virak, and Pat Morin. 2017. “Array Layouts for Comparison-Based Searching.” *Acm Journal of Experimental Algorithmics* 22 (May): 1–39. https://doi.org/10.1145/3053370.